IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 21 August 2024, accepted 24 September 2024, date of publication 3 October 2024, date of current version 14 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3472907

APPLIED RESEARCH

Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study

MARCO CANTONE, CLAUDIO MARROCCO, (Member, IEEE), AND ALESSANDRO BRIA
Department of Electrical and Information Engineering, University of Cassino and Southern Latium, 03043 Cassino, Italy

Corresponding author: Marco Cantone (marco.cantone@unicas.it)

This work was supported in part by Italian Ministry of University, Ministry of Education, University and Research (MIUR) Program "Department of Excellence" Law under Grant 232/216; and in part by the "Innovative Ph.D. Programs for Public Administration under Grant D.M.351/2022."

ABSTRACT Network Intrusion Detection Systems (NIDS) are a fundamental tool in cybersecurity. Their ability to generalize across diverse networks is a critical factor in their effectiveness and a prerequisite for real-world applications. In this study, we conduct a comprehensive analysis on the generalization of machine-learning-based NIDS through an extensive experimentation in a cross-dataset framework. We employ four machine learning classifiers and utilize four datasets acquired from different networks: CIC-IDS-2017, CSE-CIC-IDS2018, LycoS-IDS2017, and LycoS-Unicas-IDS2018. Notably, the last dataset is a novel contribution, where we apply corrections based on LycoS-IDS2017 to the well-known CSE-CIC-IDS2018 dataset. The results show nearly perfect classification performance when the models are trained and tested on the same dataset. However, when training and testing the models in a cross-dataset fashion, the classification accuracy is largely commensurate with random chance except for a few combinations of attacks and datasets. We employ data visualization techniques in order to provide valuable insights on the patterns in the data. Our analysis unveils the presence of anomalies in the data that directly hinder the classifiers capability to generalize the learned knowledge to new scenarios. This study enhances our comprehension of the generalization capabilities of machine-learning-based NIDS, highlighting the significance of acknowledging data heterogeneity.

INDEX TERMS CIC-IDS2017, cross-dataset, CSE-CIC-IDS2018, generalization, intrusion detection system, machine learning.

I. INTRODUCTION

The rapid expansion of network interconnections has led to a corresponding growth in the cyber threat landscape, attracting the interest of an increasing number of cyber attackers. This has resulted in the disruption of essential services with significant economic consequences. Globally, cybercrime is estimated to have an impact of around 1 trillion dollars in 2020, with an increase of more than 50% compared to 2018 [1]. It is therefore necessary to use systems and strategies to counter this phenomenon [2], [3], [4].

Network Intrusion Detection Systems (NIDS) are specifically designed to identify intrusions analyzing network traffic, enabling targeted entities to take timely actions against potential threats [5]. They can be grouped into three categories: statistics-based, knowledge-based, and Machine-Learning-based (ML-based) [6], [7]. The statistics-based paradigm concerns the meticulous examination of each record within a dataset, with the goal of building a statistical model that encompasses the established norms of user behavior within a network [8], [9]. In contrast, the knowledge-based approach is based on attempting to identify the requested actions by leveraging existing system data, including protocol specifications and network traffic instances. Knowledge-based NIDS, guided by human-defined rules, deduces

The associate editor coordinating the review of this manuscript and approving it for publication was Xueqin Jiang.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

typical system activities and identifies deviations from this benchmark as potential intrusions [10], [11]. A knowledge-based method to attain traffic classification is Deep Packet Inspection. This technique involves examining the content and structure of individual packets, allowing for detailed inspection of packet headers, payload data, and protocol-specific information [12].

ML-based NIDS acquire intricate pattern-matching capabilities from extensive training data either with a supervised or unsupervised approach. In the context of supervised IDS, each data instance is characterized by a pair comprising a source of network data and an associated output value, typically denoting intrusion or normal behavior. Utilizing the training dataset, a supervised learning technique is then employed to train a classifier in discerning and modeling the inherent correlation that exists between the input data and the labeled output values [13], [14], [15]. In contrast, unsupervised learning is employed to extract patterns from input datasets that lack predefined class labels. In this approach, the input data points are typically treated as a set of random variables, and a joint density model is constructed for the dataset to identify intrusions and anomalous patterns [16], [17]. ML-based-NIDS leverage a number of different methodologies including decision trees, neural networks, Support Vector Machines (SVM), and k-nearest neighbors [18], [19]. For instance, Zhou et al. [20] introduced an intrusion detection framework that employs feature selection and ensemble learning. They first utilized a heuristic algorithm, CFS-BA, for dimensionality reduction based on feature correlations. Next, an ensemble approach combined C4.5, Random Forest, and Forest by Penalizing Attributes algorithms. A voting technique was finally used to merge base learners probability distributions for attack recognition. The study of Stiawan et al. [21] investigated feature selection, aimed at improving the accuracy and reducing execution time of traffic anomaly detection in a large network dataset. Employing Information Gain, the authors identified and grouped pertinent features by considering minimum weight values. Subsequently, the selected features underwent empirical evaluation through diverse classifier algorithms, including Random Forest, Bayes Net, Random Tree, Naive Bayes, and J48, applied to the CICIDS-2017 dataset. The research by Gu and Lu [22] introduced a novel intrusion detection framework that integrates SVM with naïve Bayes feature embedding. The methodology involves applying naïve Bayes feature transformation to the original dataset and subsequently training an SVM classifier on the transformed data to construct the intrusion detection system.

More recently, also Deep Learning (DL) approaches have been adopted in the design of NIDS. DL is a subfield of ML in which deep artificial neural networks automatically learn increasingly abstract and hierarchical data representations [23], [24], [25], [26], [27]. Duan et al. [28] proposed to transform network traffic data into spatiotemporal graphs and used a dynamic line graph neural network (DLGNN) with semisupervised learning to capture evolving communication patterns. The paper by Li et al. [29] proposed an effective deep learning method, namely AE-IDS (AutoEncoder Intrusion Detection System) based on Random Forest and autoencoder. This method constructs the training set with feature selection and feature grouping using the random forest algorithm. After training, the model predicts the results with an autoencoder, which greatly reduces the detection time and effectively improves the prediction accuracy compared to traditional ML-based-IDS.

The popularity of DL for image analysis has prompted researchers to use these models for network traffic classification as well, applying them to synthetic images generated from the network traffic or a set of extracted network flows features. The literature includes research that employs both convolutional networks [30], [31], [32], [33] and vision transformers [33], [34]. The complexity of the transformer model combined with its ability to merge heterogeneous information has paved the way for a strand of research that uses it as a feature extractor and/or classifier [35], [36], [37].

In this work, we extensively analyze the generalization capabilities of ML-based-NIDS including a variety of training strategies, data sources and evaluation metrics. Generalization refers to the ability of a model to perform well on data it has never seen during training. For this investigation, we design two types of experiments: a within-dataset scenario in which we split a dataset in train and test sets, and a cross-dataset scenario where train and test sets are taken from different datasets. The datasets we utilize include CIC-IDS2017, CSE-CIC-IDS2018 [38], LycoS-IDS2017 [39], and LycoS-Unicas-IDS2018 generated for this research. These datasets are available both as raw network traffic and features-processed format computed on bidirectional flows. We utilize the available extracted features for training the models, which is the prevailing approach adopted in the literature [14], [15], [20], [21], [22], [24], [28], [29], [40], [41], [42]. The selection of CIC-IDS2017 and CSE-CIC-IDS2018 was based on the presence of similar attack classes, making them suitable for cross-dataset evaluation. We also employed LycoS-IDS2017 to investigate generalization as it offers a distinct feature set resulting from the refinement of specific features from CIC-IDS2017.

Our main contributions are:

1) **Cross-dataset analysis**: we evaluated ML-based NIDS generalization capabilities when trained and tested across 4 different datasets. The analysis was carried out encompassing a variety of ML models, feature sets, and training configurations including the discrimination between benign and malicious instances and single-attack analysis.
2) **New dataset**: we introduced LycoS-Unicas-IDS2018, an enhanced CSE-CIC-IDS2018 dataset with refined feature extraction techniques, specifically addressing the concerns highlighted by Rosay et al. [39].

3) **Explainability**: We employed ad hoc visualization techniques to enable a deeper understanding of the datasets structure and class distributions, as well as to provide visual explanations for the obtained results.

The rest of this work is organized as follows. In Section II we revise the literature regarding the datasets used and the generalization capabilities of ML-based NIDS. In Section III we describe the ML classifiers employed and the characteristics of the datasets. We also detail the experimental methodology and how we evaluate the performance. Section IV discusses the results obtained and shows some insights about the patterns present in the data. Finally, in Section V, we conclude with a summary and a critical discussion.

II. RELATED WORK

In this section, we explore research on generalization and literature works that question the construction and labeling of the CIC datasets. The discussion of generalization is enriched by an examination of federated learning (FL) approaches. FL, contrary to centralized learning, enables organizations to work together without sharing sensitive data, ensuring data privacy, compliance, and enhanced security against attacks. This decentralized approach also supports real-time adaptability and robust intrusion detection in evolving threat landscapes [43]. In Table 1 a comparison with existing literature works analyzing generalization is presented.

A. GENERALIZATION IN CENTRALIZED IDS

D'Hooge et al. [44] conducted a study on CIC-IDS2017 and CSE-CIC-IDS2018 to evaluate the performance of different ML classifiers in a cross-dataset scenario only for DoS and Botnet attacks. They concluded that the tested classifiers were not able to work properly on data from unseen networks at train time and further analysis is necessary. Verkerken et al. [45] trained four unsupervised ML models on the two CIC datasets and evaluated their performance in a cross-dataset manner. Their results indicate an average decrease of 30.45% in Area Under the Receiver Operating Characteristic curve (AUROC) during cross-dataset experiments compared to within-dataset experiments. Notably, even in the most favorable scenario, a consistent performance decrease of 17.85% in AUROC is observed for the One-Class SVM. The authors conclude that the models exhibit challenges in transferring learned information across datasets, yet they still perform better than a random classifier. In a very recent study [46], unsupervised training using PCA and autoencoders was performed on four NetFlow datasets [47]. The research evaluated generalization performance resulting in 16 experimental combinations: 4 within-dataset and 12 cross-dataset scenarios. Autoencoders demonstrated some potential for generalization since they achieved an average AUROC of 61.08% in cross-dataset evaluation, with respect to a within-dataset performance of 96.62%. In contrast, PCA obtained an average AUROC of 51.56% for cross-dataset evaluation with respect to a within-dataset performance of 95.76%. However, both methods performed worse than a random classifier in some cross-dataset configurations.

The abovementioned research sheds light on the limitations of machine learning classifiers in adapting to unseen network data, highlighting the challenges of generalization across datasets.

B. GENERALIZATION IN FEDERATED IDS

The study conducted by Popoola et al. [48] aimed to enhance generalization performance by utilizing a FL approach on four NetFlows datasets. These datasets originated from different sources but shared the same set of features [47]. The centralized models demonstrated robust performance in the within-dataset configuration but challenges arose when applied to other datasets. For instance, after training on NF-ToN-IoT-v2, the centralized model achieved an F1-score of 94.91% when tested in a within-dataset scenario. In contrast, its performance varied significantly in cross-dataset modality, with values of 78.96%, 29.19%, and 62.83% observed when tested on NF-UNSW-NB15-v2, NF-BoT-IoT-v2, and NF-CSE-CIC-IDS2018-v2, respectively. Introducing the Federated Learning approach significantly improved performance across all datasets while at the same time achieving better or comparable performance with respect to centralized models. In a study involving FL with two clients holding respectively the NF-UBSW-NB15-v2 and NF-BoT-IoT-v2 datasets, the authors explored three training scenarios [49]. They considered localized training, where only one dataset was used, centralized training, which involved training on the merged dataset, and federated training, utilizing both datasets in an FL manner. Localized training demonstrated effectiveness when trained on NF-UBSW-NB15-v2 and assessed on NF-BoT-IoT-v2 for binary-class classification, yielding an average F1-score of 87.9%. Conversely, when the models were trained on NF-BoT-IoT-v2 and tested on NF-UBSW-NB15-v2, the average F1-score was 9.36%, indicating poor generalization. The centralized approach demonstrated the highest performance, achieving an F1-score of 95.63% averaged across the various models and datasets employed. In comparison, the federated approach exhibited slightly lower performance at 91.08% F1-score. In the study of de Carvalho Bertoli et al. [50], a stacked-unsupervised federated learning approach was employed in the context of a flow-based NIDS to enhance cross-silo generalization, referring to generalization across different splits of the dataset. The authors introduced the stacked Energy Flow Classifier and an autoencoder, both trained using federated learning techniques on the 4 NetFlow datasets. Each client in this setup possessed a distinct dataset. The results were promising, demonstrating good performance on each dataset.

In all of the abovementioned literature works the FL approach involved training on data from all the available data sources, contributing to its superior cross-dataset generalization. Further, in none of these works the federated

learning models have been tested on an external dataset to fully characterize their generalization capability.

C. CIC DATASET INTEGRITY

Recently, some research investigated the integrity of the CIC datasets. Engelen et al. [51] conducted a rigorous evaluation of the CIC-IDS2017 dataset and its data collection procedures, uncovering significant issues in traffic generation, flow construction, feature extraction, and labeling. They implemented an improved data processing methodology, resulting in the reconstruction and relabeling of over 20 percent of original traffic flows. Machine learning benchmarks on the refined dataset yielded performance improvements, highlighting the impact of data collection issues on model evaluation. Rosay et al. [39] analyzed the raw traffic data of CIC-IDS2017 and the software used for feature extraction and labeling. They found different issues related to features duplication, features miscalculation, wrong protocol detection, inconsistent TCP termination flows, and error in labeling. To overcome these problems, they developed a tool to re-extract features from bidirectional flows. This yielded a corrected version of the CIC-IDS2017 dataset called LycoS-IDS2017. The study of Lanvin et al. [52] addresses deficiencies in the CIC-IDS2017 dataset labeling and traffic acquisition, including duplication and misclassified attacks. The authors contributed by presenting a refined dataset, introducing a CICFlowMeter patch, and assessing corrections on intrusion detection models. Their evaluation revealed nuanced challenges in achieving high precision without overfitting. The study underscores the critical role of dataset quality in NIDS evaluation, advocating for transparent labeling practices. Another conference paper criticizing the construction of the CIC-IDS2017 and CSE-CIC-IDS2018 datasets is that of Liu et al. [53]. Their work mainly focused on labeling issues and they concluded that labeling errors totaled 6.67% and 7.53% on CIC-IDS2017 and CSE-CIC-IDS2018 respectively, with a corruption rate that exceeded 75% for some attacks. The research by Catillo et al. [54] critically examined the limitations of public intrusion datasets and associated ML experiments, challenging the reliability of conclusions drawn in existing literature despite perceived progress. Although academic enthusiasm and increasing complexity in machine and deep learning, the practical effectiveness of public datasets for real-world network intrusion detection is questioned. The paper emphasizes the necessity for caution in handling the collection, release, and utilization of existing intrusion datasets in research. It also asserts that genuine advances in NIDS are hampered by numerous flaws and limitations, supported by scientific insights from existing datasets, related literature, and direct field experience.

III. MATERIALS AND METHODS

A. DATASETS

In this study, we employ four different datasets: CIC-IDS2017, CSE-CIC-IDS2018, LycoS-IDS2017 e LycoS-Unicas-IDS2018. The first two are from the Canadian Institute for Cybersecurity (CIC), whereas the other two are constructed from the same raw traffic of the CIC datasets but have slightly different features and correct some of the problems mentioned in Section II-C. These datasets exhibit same features and similar attack classes, with several overlapping attack types, making them the perfect candidates for a generalization analysis. The size and variety of attack types were also an important factor in selecting the datasets to use. We discarded the NetFlow datasets proposed in [47] since they do not share attack classes. Other datasets [55], [56], [57], [58] were not considered because they were collected in contexts very different from computer networks (such as electric vehicle charging infrastructure, video surveillance networks, or IoT environments) or because they were limited to a single type of attack (DDoS).

In Table 2 is reported the class composition for all the datasets. It is worth noting that DDoS LOIC-UDP and DDoS HOIC attacks are absent in the 2018 datasets, whereas PortScan and Heartbleed are not present in the 2017 datasets. Furthermore, there is a significant class imbalance between the benign and multiple attack classes, specifically the benign class is predominant in all datasets, comprising 80.32% (CIC-IDS2017) 75.96% (LycoS-IDS2017) 82.98% (CSE-CIC-IDS2018) and 73.04% (LycoS-Unicas-IDS2018).

1) CIC-IDS2017

CIC-IDS2017 (CIC17) is an intrusion detection dataset from the Canadian Institute for Cybersecurity [38]. The dataset includes 7 days of network traffic acquired from an infrastructure consisting of 14 machines, where several cyber attacks were carried out. Benign background traffic was generated using a system called B-Profile, which extracted abstract behavior from a group of real users through a combination of machine learning and statistical analysis techniques. In addition, features suitable to machine learning are available in CSV files, one for each day of acquisition. These features were extracted using the CICFlowMeter software through distinct phases, including the identification of bidirectional flows, feature extraction, and labeling. Overall the dataset consists of 84 features and 15 classes of which 14 refer to cyber-attacks and one represents benign traffic.

Since singular values (*NaN* and *infinite*) were present in some features, a data cleaning step was needed to remove all the samples involved. For our study, we excluded the following features: (i) `Flow ID`, `Source IP`, `Source Port`, `Destination IP`, and `Protocol` due to their role as flow identifiers; (ii) `Timestamp` as it could bias the model during training; and (iii) `Fwd Header Length` since it was redundant.

2) CSE-CIC-IDS2018

The Canadian Institute for Cybersecurity released a second dataset in collaboration with the Communication Security

TABLE 1. Comparison between the generalization analysis carried out in this research vs. other literature works.

	Datasets			Cross-dataset	Single-attack	Normal vs. malicious
	CIC	LycoS	NetFlow			
D'hooge et al. [44]	✓	✗	✗	✓	✓	✗
Verkerken et al. [45]	✓	✗	✗	✓	✗	✓
Sičić et al. [46]	✗	✗	✓	✓	✗	✓
Popoola et al. [48]	✗	✗	✓	✗	✗	✓
Sarhan et al. [49]	✗	✗	✓	✗	✓	✓
de Carvalho Bertoli et al. [50]	✗	✗	✓	✗	✗	✗
Our Work	✓	✓	✗	✓	✓	✓

Establishment (CSE) known as CSE-CIC-IDS2018 (CIC18). This dataset utilized the same B-Profiles methodology for generating benign traffic. Notably, the second dataset expanded the generation process to a larger scale by migrating the entire network setup to the Amazon Web Services (AWS) cloud computing platform and deploying a total of 500 machines, a substantial increase with respect to the 14 utilized in the first iteration.

Similar to CIC-IDS2017, CSE-CIC-IDS-2018 is distributed in two formats: raw network packets (PCAPs) and machine learning-friendly CSV files that encapsulate bidirectional flows derived from the PCAPs through the application of CICFlowMeter. It is noteworthy that the CSV files in this instance lack four features related to flow identification. For consistency with CIC-IDS2017, we removed the `Timestamp` and `Protocol` features, and applied the same data cleaning.

3) LycoS-IDS2017

In response to identified anomalies within the CIC-IDS2017 dataset, researchers undertook a meticulous correction process, resulting in the creation of the LycoS-IDS2017 (LycoS17) dataset [39]. The comprehensive analysis of the CIC dataset revealed several issues, including feature duplication and miscalculations. Notably, instances of redundancy were observed in features such as `Average Packet Size` and `Packet Length Mean`, which shared identical semantic meanings. Furthermore, discrepancies arose from the incorrect calculation of numerous features due to type-related oversights and updating errors concerning the directionality of flow. The protocol detection mechanism in the original dataset exhibited shortcomings, leading to misclassification of ICMP and other protocol packets. Additionally, inconsistencies in TCP termination flows and uncertainties surrounding the labeling process prompted the development of remedial measures. To address these concerns, a tool called LycoStand was developed to accurately extract a set of features based on the predefined feature set of the CIC datasets.

Following the process of feature extraction, a Python-based labeling script was employed, referencing data from the official CIC website that had been refined through meticulous analysis conducted using Wireshark. Noteworthy is the decision to discard an entire afternoon of acquisition data, a precautionary measure taken due to substantial doubts regarding the reliability of labeling.

4) LycoS-Unicas-IDS2018

LycoS-Unicas-IDS2018 (LycoS18) is a novel dataset created by us using LycoStand [39], a feature extractor of raw network traffic, on the PCAP files of CSE-CIC-IDS2018, and labeling the flows according to the official attack schedule accessible via the dataset website. While certain anomalies have been identified in the annotation of the CIC-IDS2017 dataset, achieving an optimal level of labeling precision remains an intricate challenge. Nevertheless, given the currently available information, the existing labeling methodology represents the most viable approach to address this concern. For the same reason we discarded raw network traffic on days 28/02/2018 and 01/03/2018 containing the infiltration attack, as it has been done for LycoS-IDS2017. After feature extraction, we obtained 46,950,511 benign samples, so we randomly subsampled this class to 10 million samples both to mitigate the imbalance between classes and for reasons related to computational complexity. The LycoS-Unicas-IDS2018 dataset is publicly available at https://github.com/MarcoCantone/LycoS-Unicas-IDS2018.

B. ML MODELS

1) LINEAR DISCRIMINANT ANALYSIS

Linear Discriminant Analysis (LDA) is a statistical technique utilized for both dimensionality reduction and classification. As a classifier, it represents one of the simplest methods, functioning by maximizing the separation between multiple classes by projecting the feature space onto a lower-dimensional subspace. The algorithm achieves this by determining the linear combinations of features that best discriminate between classes while minimizing within-class variance. During inference, LDA assigns a class to the

TABLE 2. Distribution of class instances for all the datasets employed.

	CIC-IDS2017	CSE-CIC-IDS2018	LycoS-IDS2017	LycoS-Unicas-IDS2018
Benign	2,271,320	13,390,249	1,395,675	10,000,000
DoS GoldenEye	10,293	41,508	6,765	26,861
DoS Slowloris	5,796	10,990	5,674	10,274
DoS Hulk	230,124	461,912	158,988	1,802,966
DoS Slowhttptest	5,499	139,890	4,866	105,550
DDoS LOIC-HTTP	128,025	576,191	95,683	289,328
DDoS LOIC-UDP	0	1,730	0	2,382
DDoS HOIC	0	686,012	0	1,074,379
SSH-Patator	5,897	187,589	2,959	92,648
FTP-Patator	7,935	193,354	4,003	190,300
PortScan	158,804	0	160,106	0
Bot	1,956	286,191	735	96,154
Infiltration	36	160,639	0	0
Web Attack - Brute Force	1,507	611	1,360	260
Web Attack - XSS	652	230	661	116
Web Attack - Sql Injection	21	87	12	50
Heartbleed	11	0	11	0

input sample based on the learned discriminant functions. We opted for LDA due to its simplicity, facilitating a meaningful comparison with other powerful models and for comprehending the learning difficulty of this task.

2) DECISION TREE

A Decision Tree (DT) is a hierarchical model used in machine learning for tasks like classification or regression. It makes decisions based on the values of features, recursively dividing the data into subsets. The construction involves selecting features at each node to optimize criteria like information gain or variance reduction. The process continues until a stopping criterion is met. Once built, the tree can be used to classify or predict new data by traversing from the root to a leaf node. Decision trees are interpretable but can overfit, so techniques like pruning or ensemble methods are often used for better generalization.

3) RANDOM FOREST

Random Forest (RF) is a bagging ensemble of decision trees to improve predictive accuracy and control overfitting. It operates by constructing a multitude of decision trees during training and outputs the average prediction. Each tree is trained on a random subset of the data and may use a random subset of features at each split, introducing diversity and enhancing robustness. Random Forest has been widely adopted in the context of NIDS both for feature selection and classification, also in combination with other models [29], [41], [59], [60].

4) XGBoost

XGBoost [61] (XGB), short for eXtreme Gradient Boosting, is a boosting ensemble learning algorithm designed for classification and regression tasks. It operates by sequentially building a series of decision trees and combines their predictions to achieve superior accuracy. XGB employs a gradient boosting framework, optimizing model performance by minimizing the gradient of the loss function. It integrates regularization techniques to control model complexity and prevent overfitting and utilizes a strategy to iteratively correct errors made by preceding trees. The algorithm excels in handling large datasets and is renowned for its efficiency, scalability, and ability to capture complex relationships within the data, making it a prominent choice in diverse ML applications. XGB has been employed in many works concerning applications related to network traffic classification [62], [63], [64].

C. EXPERIMENTAL METHODOLOGY

The focus of this work is investigating the generalization capabilities of NIDS based on ML using different datasets for training and evaluating the models. This resembles real-world scenarios wherein these systems must operate. For this purpose, we designed two major categories of experiments: *within-dataset* and *cross-dataset*. Moreover, to understand the motivation behind the discrepancy between within-dataset and cross-dataset results, we conducted additional experiments, utilizing samples from individual attack classes separately and varying the number of features.

1) WITHIN-DATASET VS. CROSS-DATASET

We conducted within-dataset experiments to assess the model performance on a designated split of the training dataset. This evaluation provides insights into the models ability to discern patterns from a single source dataset and establishes a reference performance. In cross-dataset experiments, the model is evaluated on a dataset that originates from a network entirely distinct from the one that provided the training dataset. This analysis aims to ascertain the generalization capabilities of the model, determining its effectiveness in

TABLE 3. Hyperparameter search space for each classifier.

Classifier	Hyperparameter	Values
LDA	solver	["svd", "lsqr", "eigen"]
	shrinkage	[None, "auto", 0.1, 0.5, 0.9]
DT	criterion	["gini", "entropy"]
	splitter	["best", "random"]
	max_depth	[None, 20]
	min_samples_split	[2, 4, 8, 16]
	min_samples_leaf	[1, 2, 4]
	max_features	[None, "sqrt", "log2"]
	max_leaf_nodes	[None, 10,000, 1,000,000]
RF	criterion	["gini", "entropy"]
	max_depth	[None, 10]
	min_samples_split	[4, 16]
	max_features	[None, "sqrt", "log2"]
	n_estimators	[10, 50]
XGBoost	max_depth	[3, 6, 12]
	n_estimators	[10, 50]
	learning_rate	[0.1, 0.3, 1]
	booster	["gbtree", "gblinear"]
	min_child_weight	[0.5, 1, 2]
	gamma	[0, 1, 10]

applying the learned patterns to novel data sources. The training is conducted using a binary classification approach, i.e. grouping all attacks labels with a single malicious label, allowing us to assess the model capability to distinguish normal traffic from anomalous traffic.

2) SINGLE-ATTACK

Contrary to grouped binary classification training, where all attack classes are grouped under a single label, we performed also *single-attack* experiments, utilising samples pertaining to a singular class of attack, alongside samples representing benign traffic. This approach provides insights into the models ability to generalize across datasets while considering the nature and modalities of specific attacks and the correlation between attack type and generalization. Since some attacks are not present in both the datasets employed in each cross-dataset experiment, we consider only the following attacks: DoS GoldenEye, DoS Slowloris, DoS Hulk, DoS Slowhttptest, DDoS LOIC-HTTP, SSH-Patator, FTP-Patator, Bot, Infiltration, Web Attack Brute Force, Web Attack - XSS and Web Attack - Sql Injection. In addition, single-attack experiments for the Infliltration class are performed solely with CIC datasets since this attack is absent in the LycoS datasets. Regarding DDoS attacks, we maintain exclusively DDoS LOIC-HTTP samples because, although datasets from 2018 have 3 types of DDoS attacks, CIC17 and LycoS17 comprise only this kind of DDoS. To address high class imbalance, we undersampled the benign class to maintain a 10:1 ratio between benign and attack traffic. This decision was made to trade-off the detrimental effects of heavily imbalanced classes with an adequate representation of the predominant benign class. To decrease the performance variability resulting from undersampling, we performed each single-class experiment three times.

TABLE 4. Performance obtained for each classifier for all the within-dataset and cross-dataset configurations tested.

Train set	Test set	Classifier	MCC	F1	AUROC
CIC17	CIC17	LDA	64.86%	69.02%	96.45%
		DT	99.72%	99.78%	99.97%
		RF	99.74%	99.79%	99.98%
		XGB	99.74%	99.79%	100.00%
	CIC18	LDA	32.62%	39.12%	81.70%
		DT	30.82%	31.20%	59.14%
		RF	25.96%	25.76%	66.93%
		XGB	35.68%	26.43%	80.44%
CIC18	CIC18	LDA	76.52%	79.82%	94.13%
		DT	96.33%	96.89%	98.85%
		RF	96.45%	96.99%	98.87%
		XGB	96.48%	97.02%	99.10%
	CIC17	LDA	39.29%	48.41%	85.06%
		DT	35.50%	33.86%	57.80%
		RF	44.44%	51.21%	75.77%
		XGB	41.75%	44.58%	79.15%
LycoS17	LycoS17	LDA	94.14%	95.56%	99.54%
		DT	99.83%	99.87%	99.92%
		RF	99.89%	99.92%	100.00%
		XGB	99.87%	99.90%	100.00%
	LycoS18	LDA	-15.32%	10.24%	57.24%
		DT	8.55%	17.04%	53.11%
		RF	9.29%	13.97%	65.46%
		XGB	10.17%	14.42%	61.54%
LycoS18	LycoS18	LDA	91.32%	93.44%	99.78%
		DT	99.77%	99.83%	99.99%
		RF	99.76%	99.83%	99.99%
		XGB	99.67%	99.76%	99.96%
	LycoS17	LDA	60.41%	70.46%	91.66%
		DT	1.24%	2.56%	50.15%
		RF	38.33%	37.29%	77.49%
		XGB	40.29%	35.54%	81.37%

3) FEATURE SELECTION

In the field of ML, feature selection is often used to mitigate overfitting and enhance generalization. For this reason, we employed the mRMR (minimum Redundancy Maximum Relevance) [65], [66] technique. This method involves identifying an optimal subset of features by minimizing redundancy among selected features while maximizing their relevance to the target variable, ensuring the models efficiency and effectiveness in capturing relevant information. It evaluates the mutual information between each feature and the target variable and, at the same time, it considers the redundancy between features. Our decision to employ mRMR was driven by two primary motivations. Firstly, we opted for a method of feature selection rather than reduction to maintain explainability of the features. By selectively choosing features, we can gain valuable insights into

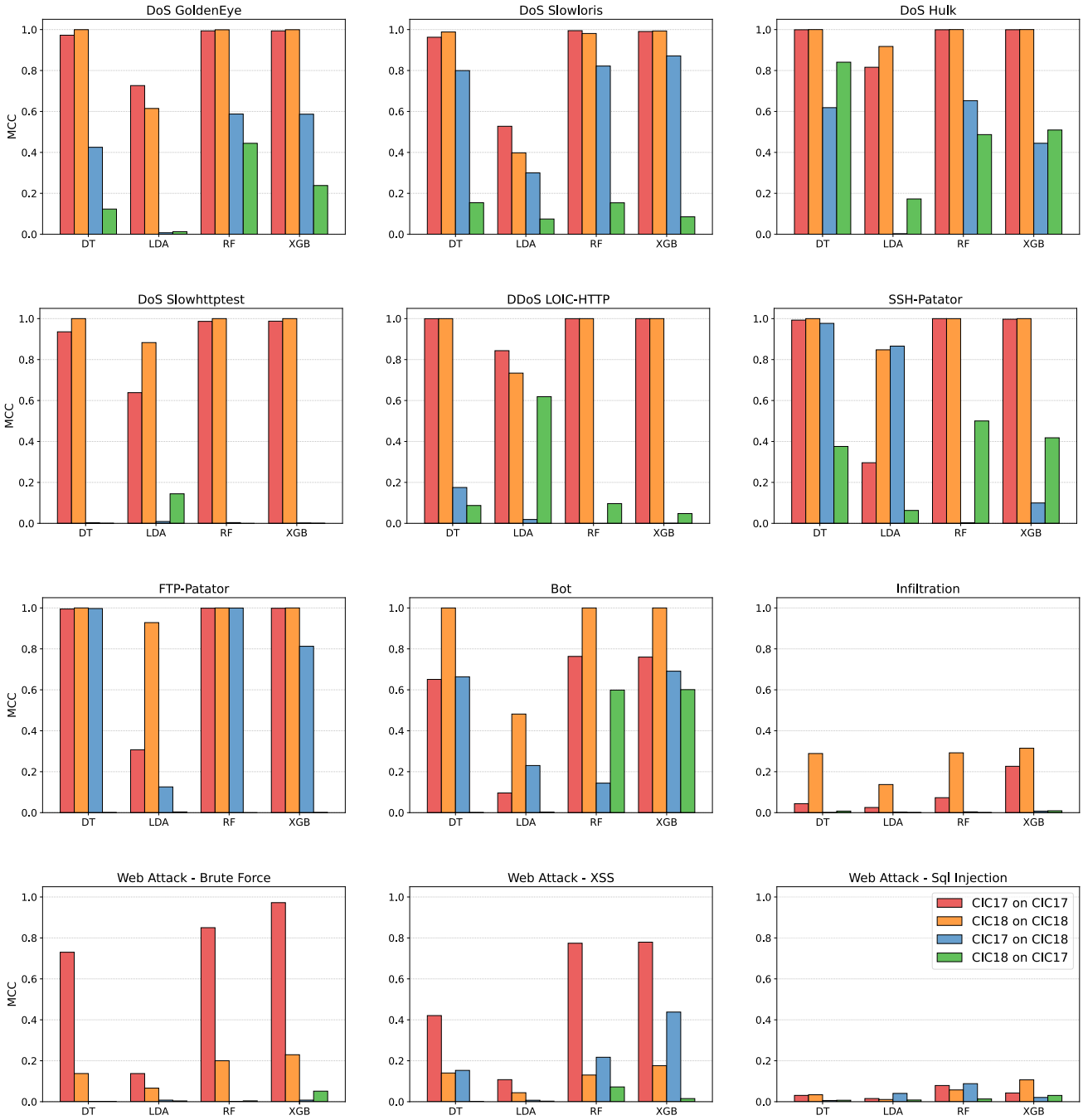


FIGURE 1. **MCC value obtained for each classifier and train-test combination with CIC datasets in single-attack experiments.**

which aspects of the data are most influential in predicting the target variable, thus enabling visual feature space analysis. Secondly, many features are highly interdependent by construction (e.g. `fwd_pkt_len_mean` is determined knowing `fwd_pkt_len_tot` and `fwd_pkt_cnt`), hence the choice of mRMR that selects the best set of n features by assessing both relevance and redundancy, resulting in a balanced selection of features that are informative but not redundant. We conducted grouped binary experiments, testing different numbers of features, specifically 1, 2, 3, 4, 5, 10, and 20. Additionally, we performed single-attack experiments using only the two top features.

4) VISUAL FEATURE SPACE ANALYSIS

We conducted an analysis of LycoS datasets structure using visualization techniques to better understand the obtained results. First, we determine the number of unique values

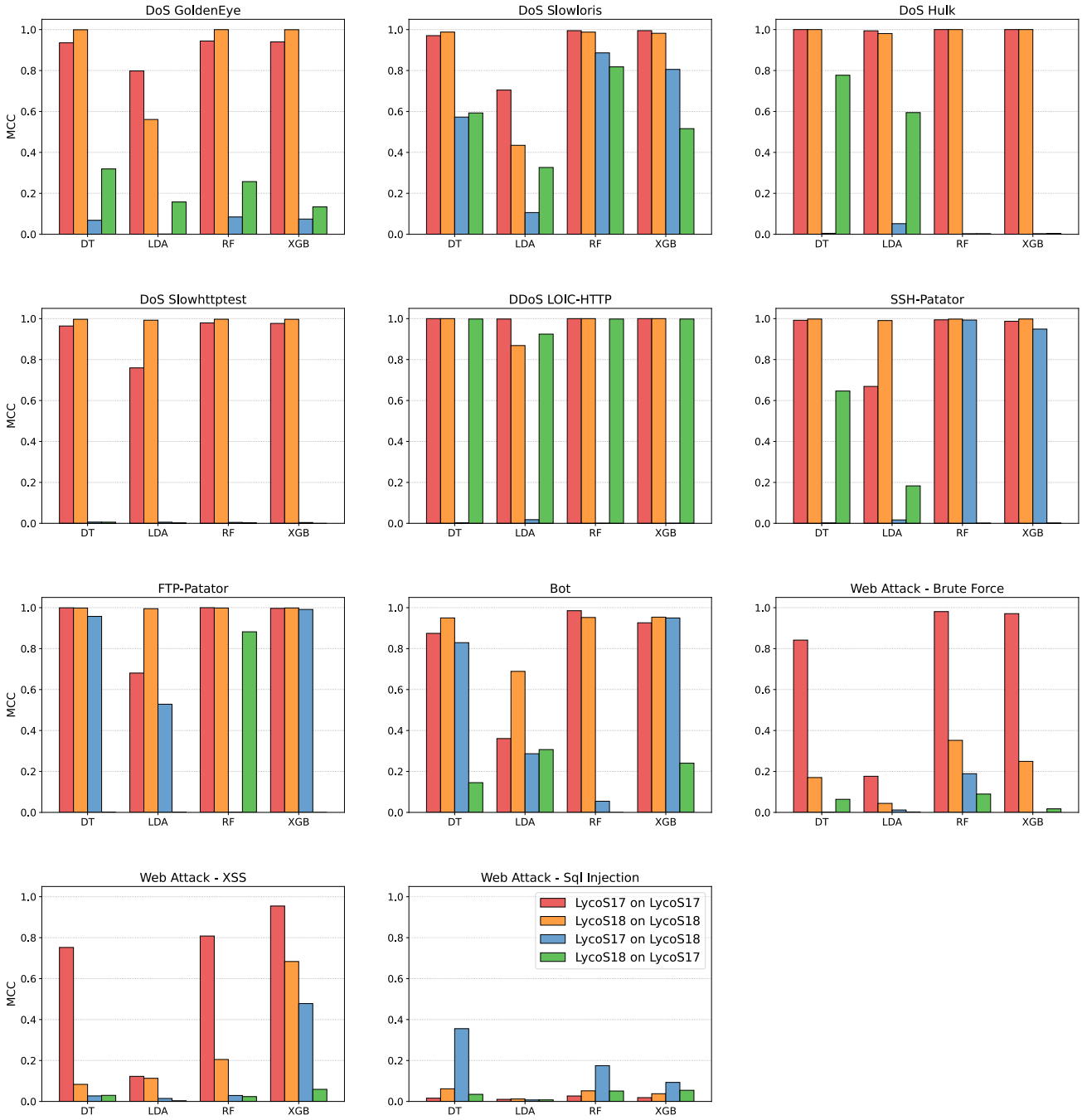


FIGURE 2. **MCC value obtained for each classifier and train-test combination with LycoS datasets in single-attack experiments.**

for each of the two best features obtained with mRMR for every attack class. This approach yields a quantitative measure of sample redundancies within the two datasets. Then, we employ Kernel Density Estimation (KDE) to analyze qualitatively the differences among the attacks distributions between the two datasets in the aforementioned 2D feature subspace. We adopt the implementation of KDE provided by the Seaborn library [66]. Where one of the two features exhibits zero variance or perfect covariance, the employed plotting function fails. Consequently, in these cases, we utilize a scatter plot to effectively depict the distribution. To further assess diversity in classes distribution between LycoS17 and LycoS18 datasets, we compute the KDE estimation of attacks distributions in a 2D subspace obtained with Principal Component Analysis (PCA) fitted on LycoS17. This technique projects the samples in a subspace where each feature is a linear combination of the original features. Thus, contrary to using only two features, PCA

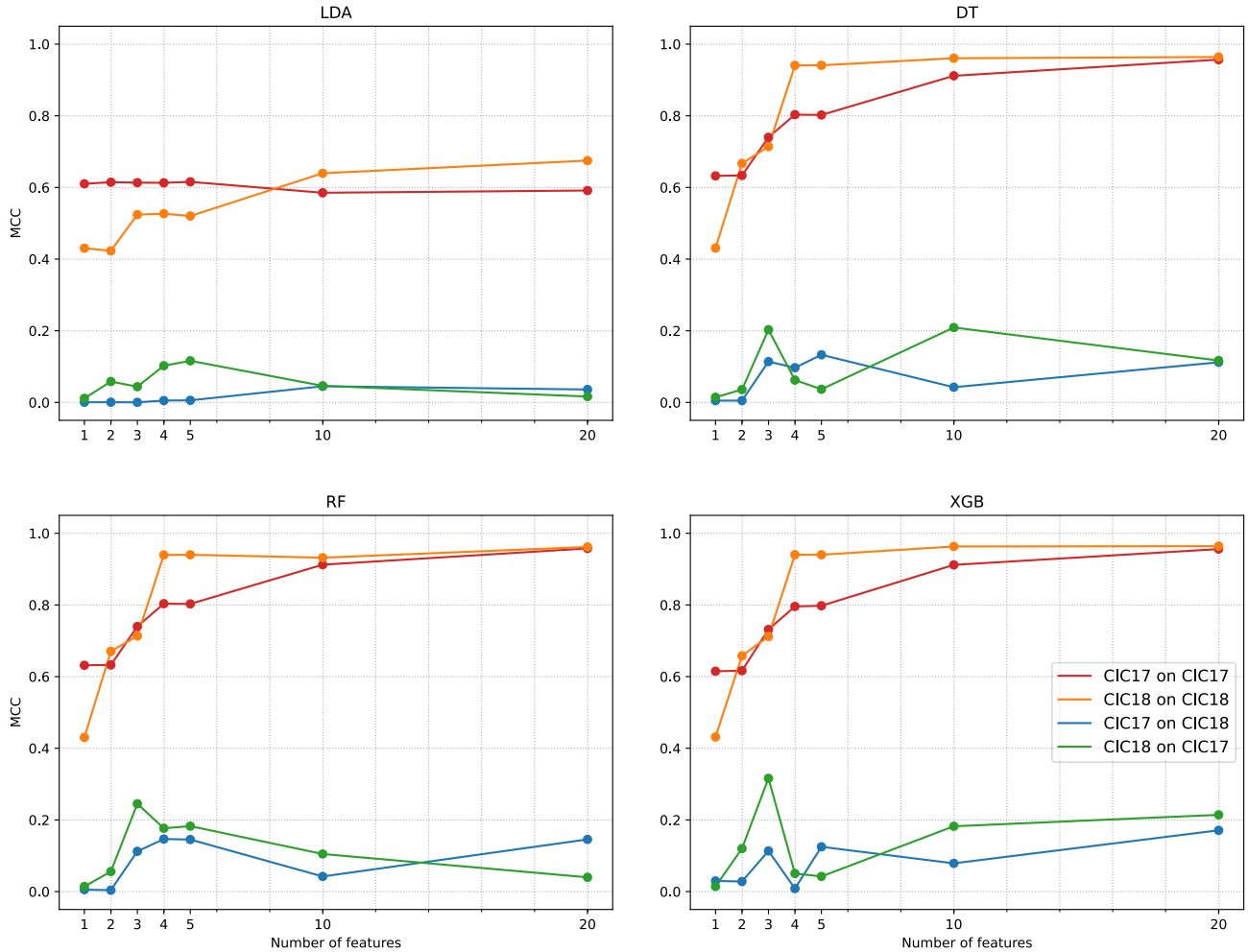


FIGURE 3. MCC for each classifier varying the number of features for the 4 train-test set combination of CIC datasets.

produces a condensed representation mixing the information from all the features.

D. TRAINING STRATEGY

In the within-dataset experiment, the dataset was randomly split according to an 80:20 ratio in train and test sets. Cross-dataset experiments utilized one entire dataset for training and another for evaluating the model performance. Before feeding the data to the model, we applied MinMax normalization on each feature. In addition, only for LycoS datasets the categorical feature `Ip Prot` is encoded using a one-hot encoder, with categories representing less than 0.1% frequency mapped into a single feature. A grid search was adopted for selecting the best hyperparameters for each classifier on a 20% random subset of the training data. The hyperparameter space for each classifier is reported in Table 3. When the optimal hyperparameter configuration is determined, the classifier is retrained on the entire training set.

E. PERFORMANCE EVALUATION

For evaluating model performance we computed the Matthews Correlation Coefficient (MCC), the F1-score and the Area Under the Receiver Operating Characteristic (AUROC) curve. The MCC [67], [68], [69] is a correlation coefficient between classifier output and true label, hence it yields a more informative and truthful score in evaluating binary classifications than accuracy, especially in the case of unbalanced datasets. The MCC is defined as:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

The F1-score represents the harmonic mean between precision and recall. Precision measures the accuracy of positive predictions, while recall measures the ability of the model to correctly identify all positive instances in the dataset. The harmonic mean combines these two measures into a single value, making it a useful indicator of model performance. The

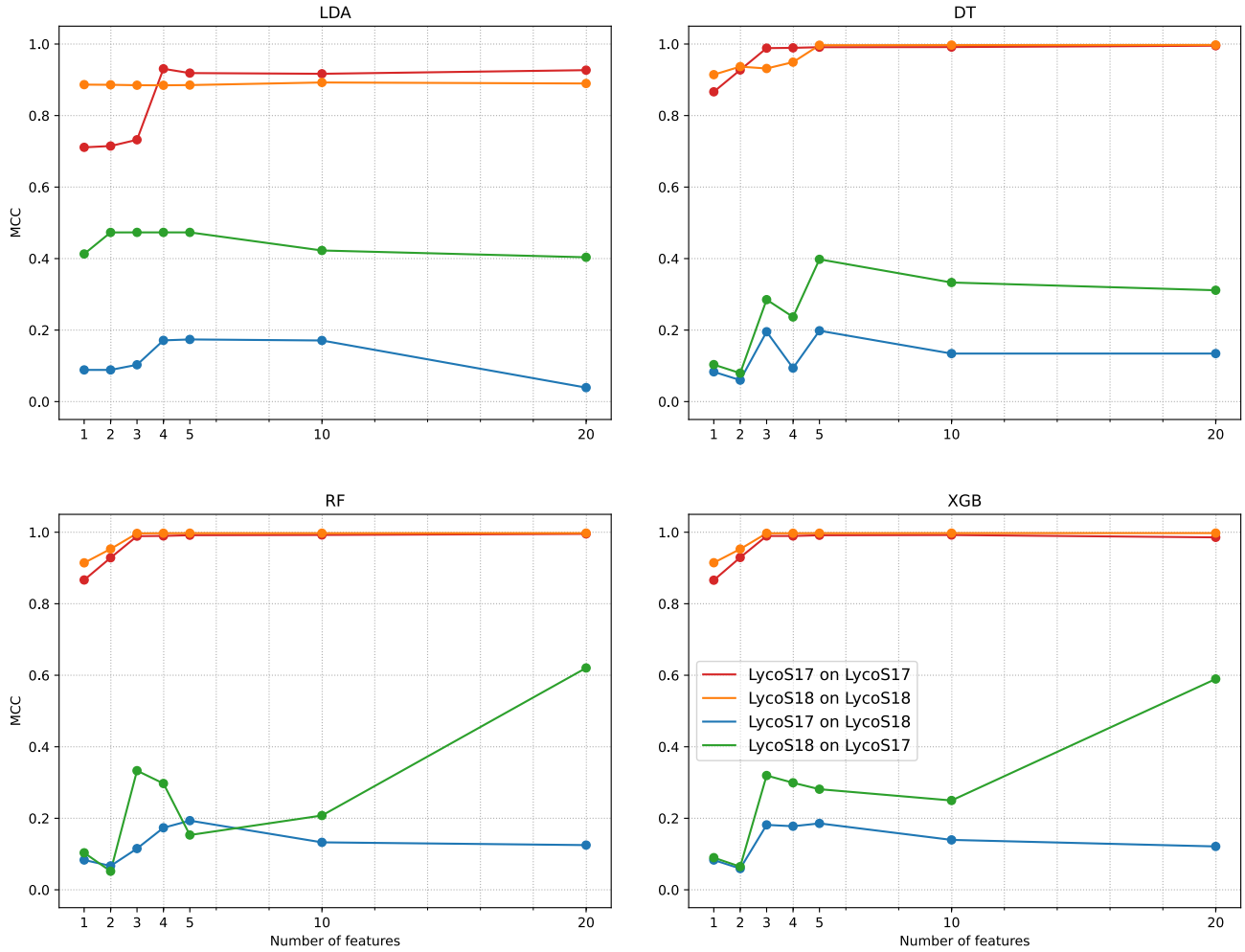


FIGURE 4. MCC for each classifier varying the number of features for the 4 train-test set combination of LycoS datasets.

F1-score is defined as:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

where:

- $Precision = \frac{TP}{TP+FP}$
- $Recall = \frac{TP}{TP+FN}$

The Receiver Operating Characteristic (ROC) curve shows the prediction ability of a binary classifier by representing the sensitivity over the false-positive rate varying the threshold value. The area under the ROC curve is a widely used metric that offers a simple way to summarize the overall performance of a model. We decided not to report the accuracy value since, given the datasets inherent imbalance, reporting accuracy might be misleading, especially as a classifier outputting only the most frequent value can yield a seemingly high performance. For instance, 11 out of 14 attacks present in CIC17 have a benign-attack ratio greater than 200, resulting in a 99.5% accuracy for a model predicting always the Benign class.

F. EXPERIMENTAL ENVIRONMENT

The code used for training and evaluating ML models was developed using Python 3.8.10 and the Scikit-learn framework version 1.3.0. All the experiments were performed on a machine running Ubuntu 18.04.5 LTS, equipped with 4 Intel(R) Xeon(R) CPU E5-4610 v2 @ 2.30 GHz and 256 GB of RAM.

IV. RESULTS AND DISCUSSION

A. WITHIN-DATASET VS. CROSS-DATASET

Table 4 reports the results obtained for the 4 within-dataset and 4 cross-dataset experiments for each classifier employed. Within-dataset experiments show significantly superior performance compared to cross-dataset ones, with an average MCC among the diverse classifier and train-test set combinations of 94.63% and 29.35%, respectively. It follows that neither of the classifiers was able to transfer the learned patterns when evaluated on a new dataset originating from a different network than the one used for training. The cross-dataset combination that obtained the worst result was the

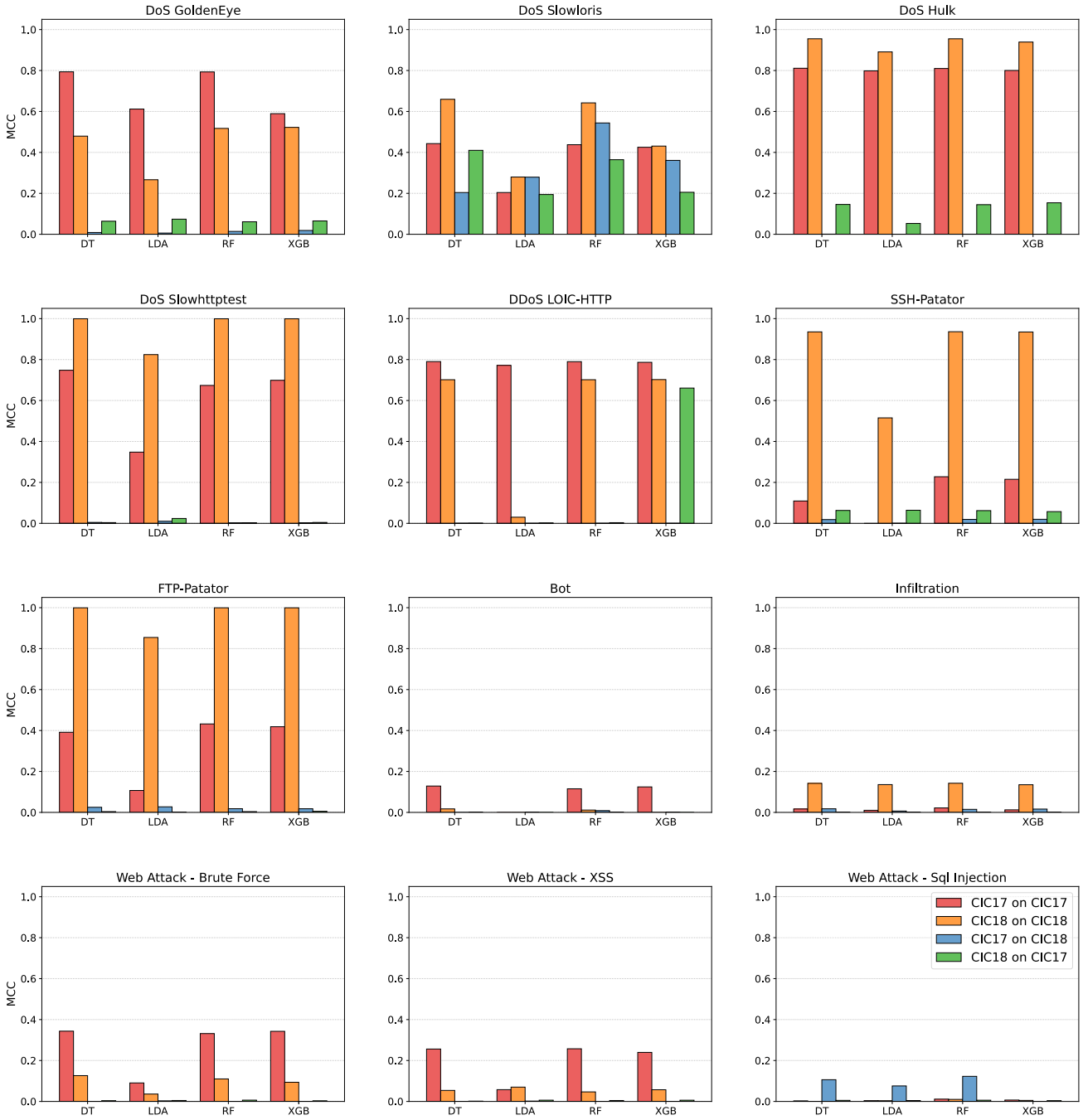


FIGURE 5. MCC value obtained for each classifier and train-test combination with CIC datasets in single-attack experiments using the best two features selected with mRMR.

LycoS17 on LycoS18 with an average MCC of 10.83%, while the best result was obtained by CIC18 on CIC17 with an average value of 40.24%. This is likely due to a too homogeneous class representation that is very simple to learn in within-dataset experiments but not sufficient to generalize on an external dataset. In support of this hypothesis, 9 out of 16 within-dataset experiments achieved near-perfect classification performance with an MCC of over 99.6%. This is unusual in many ML problems and can be a consequence of low variability in the training datasets and sample redundancies.

When trained on CIC18, neither of the four classifiers achieved a within-dataset MCC greater than 97%, while for all the other within-dataset combinations many models easily reached and surpassed a value of 99%. In the cross-dataset scenario, the average MCC during training on CIC18

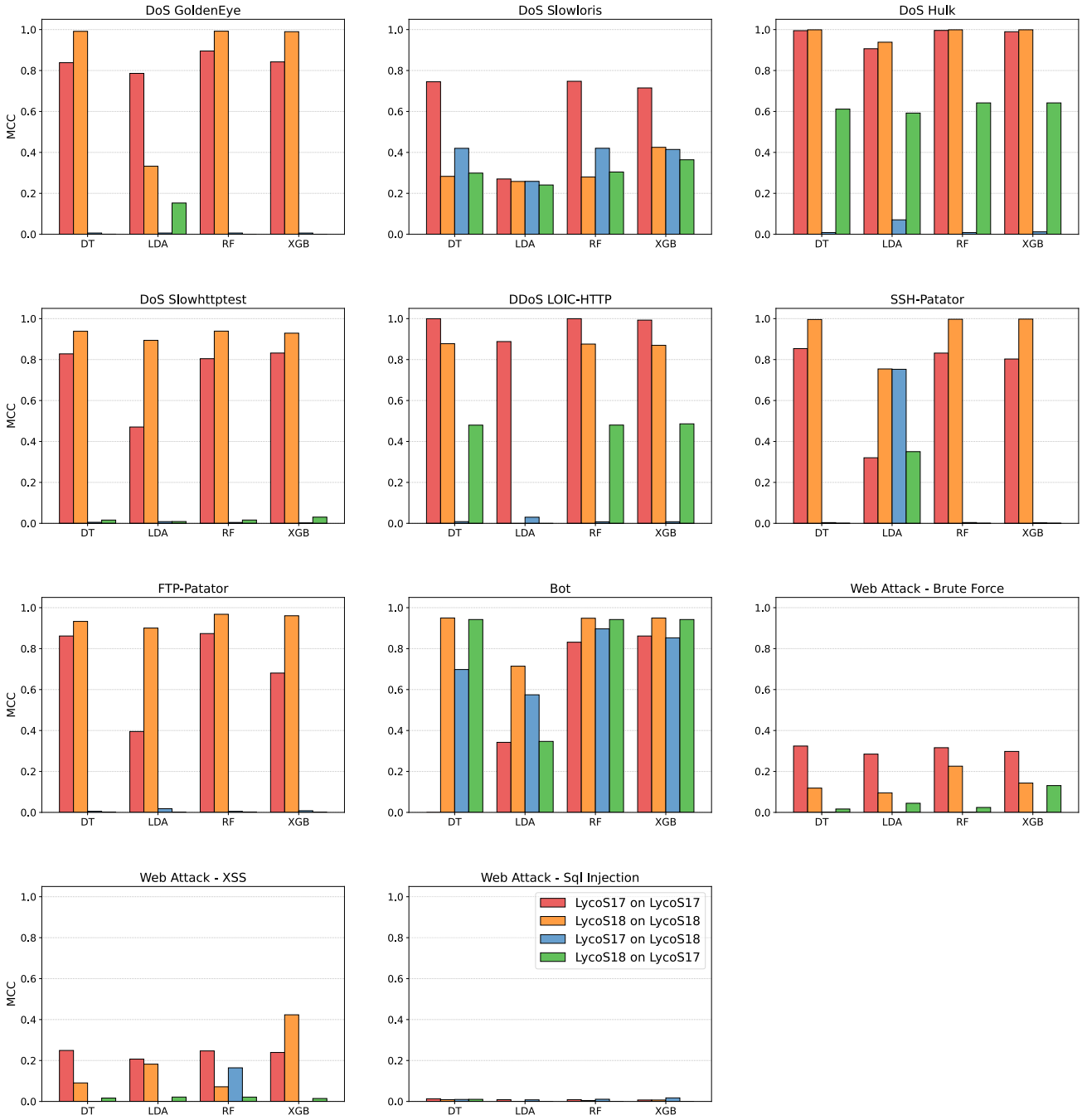


FIGURE 6. **MCC value obtained for each classifier and train-test combination with LycoS datasets in single-attack experiments using the best two features selected with mRMR.**

was 40.24%, while training on the CIC17 dataset yielded a lower average of 31.27%. Consequently, pretraining on CIC18 yielded suboptimal within-dataset results but demonstrated superior performance in cross-dataset evaluation. This observed difference may be attributed to the larger dimensionality of the CIC18 dataset, which potentially provides a more comprehensive representation of network traffic, encompassing 13,691,268 instances compared to the 1,837,498 instances of the CIC17 dataset.

Among all classifiers, LDA is the one that obtained the worst results in the within-dataset experiments whereas in cross-dataset experiments, instead, its performances are aligned with other classifiers and achieved the best result among all the cross-dataset experiments when trained on LycoS18 and evaluated on LycoS17 with a 60.41% MCC. This can be due to the lower model complexity that reduces overfitting and thus benefits generalization.

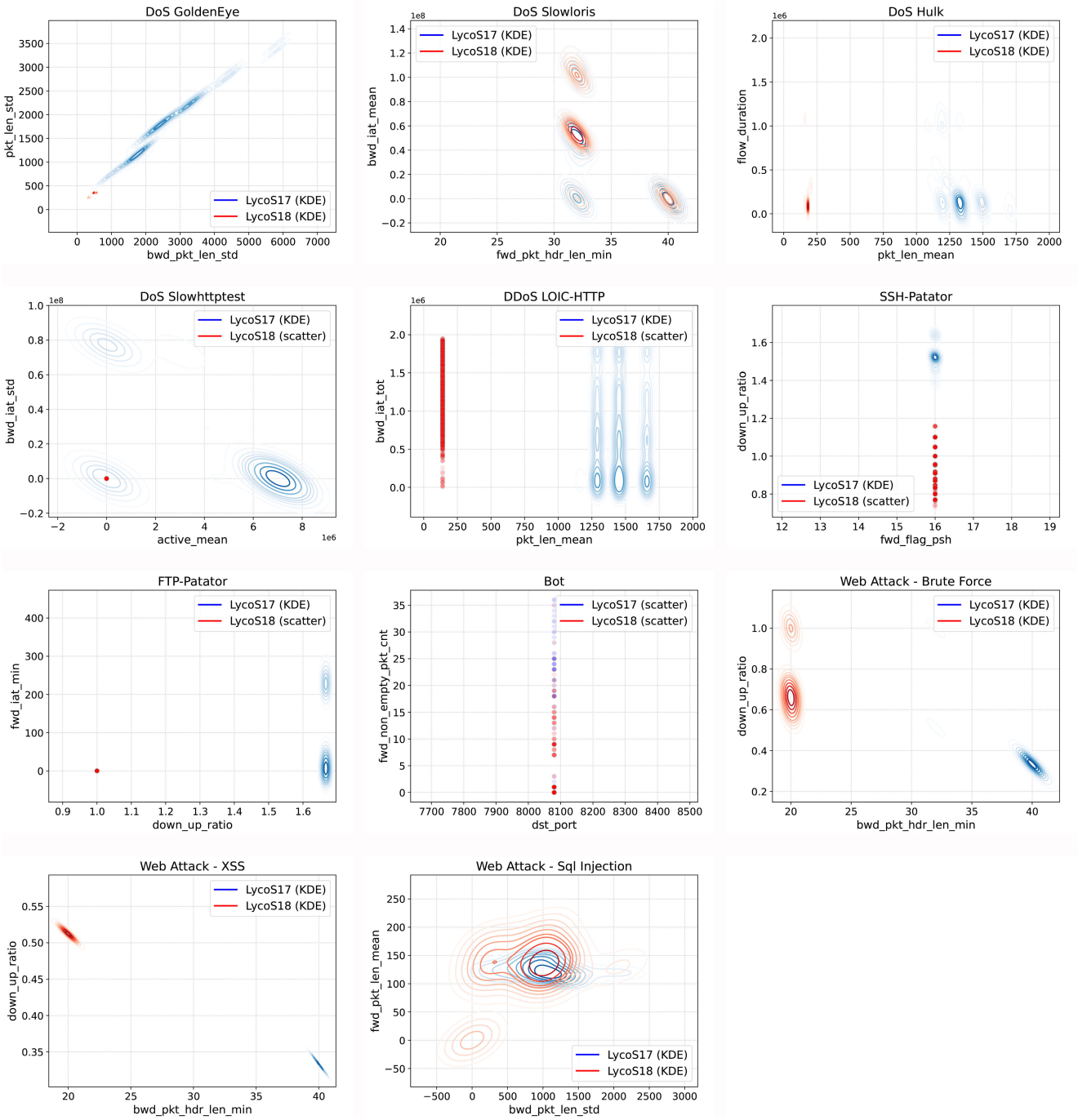


FIGURE 7. Attacks distribution of the LycoS17 and LycoS18 datasets in the subspace of the best two features obtained with mRMR using Lycos2017 as training set. The legend, in parentheses, shows the type of representation employed to visualize the distribution.

B. SINGLE-ATTACK

Figure 1 and Figure 2 present the results obtained by single-attack experiments on CIC and LycoS datasets, respectively. For each experiment we report the highest MCC value across the three runs (see Section III-C2). It can be observed that there was no classifier able to generalize the patterns of a specific attack across all combinations of train and test sets.

There are 7 attacks for which the classifiers exhibited high performance in within-dataset experiments, namely DoS GoldenEye, DoS Slowloris, DoS Hulk, DoS Slowhttptest, DDoS LOIC-HTTP, SSH-Patator and FTP-Patator. Other 4 attacks generally showed weak performance in within-dataset evaluation, namely all 3 web attacks and Infiltration. This is due to the very low number of samples that made it difficult the learning process. Regarding the Bot attack, the

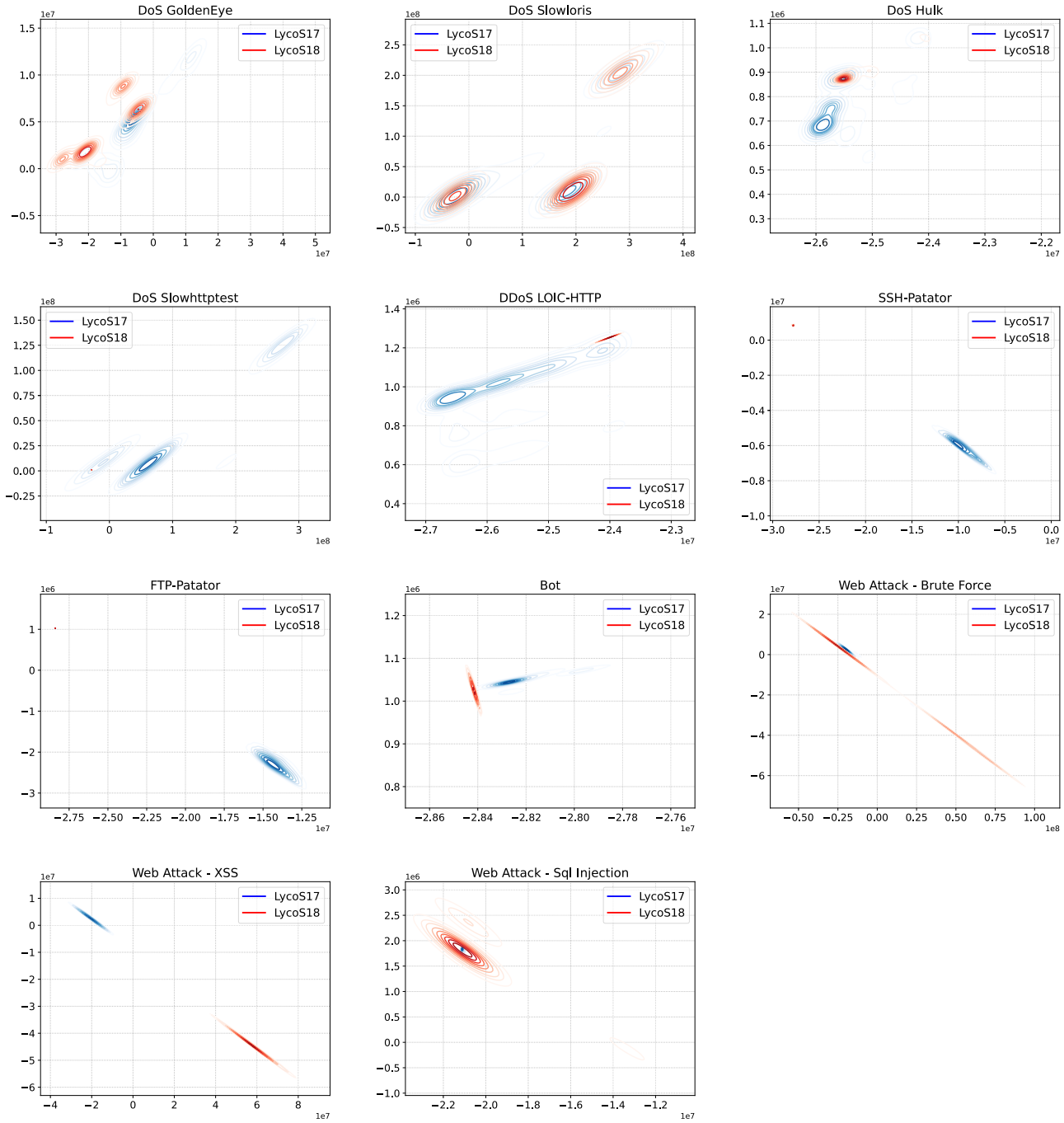


FIGURE 8. KDE estimation of the attacks distribution for the LycoS17 and LycoS18 datasets in the feature subspace obtained through PCA fitted with attack and benign samples from LycoS17.

results are strongly dependant on the specific dataset used for training. When the models are trained and evaluated on CIC18, the performances are near 100% MCC, whereas if CIC17 is chosen as training set the MCC value dropped slightly above 60%. When considering the LycoS datasets, this discrepancy is not present and the within-dataset results when training on LycoS17 and LycoS18 are similar. This suggests that the approach of [39], that we adopted to generate LycoS18, has probably corrected the issues present in the CIC datasets.

Cross-dataset experiments showed some generalization capabilities depending on the attack type and also on the specific train-test combination used for cross-dataset evaluation. For the following attacks, DoS GoldenEye, DoS Slowloris, DoS Hulk, DDoS LOIC-HTTP, SSH-Patator, FTP-Patator and Bot, 40 out of 112 combinations of classifier and

train-test datasets showed MCC superior to 50%, indicating partial transfer of the learned patterns into the new dataset. In the case of DoS Slowhttptest, Infiltration, Web Attack - Brute Force, Web Attack - XSS and Web Attack - Sql Injection, none of the classifiers in any of the train-test combinations were able to achieve MCC greater than 50%, and in many cases, the performances are almost zero. For the attack FTP-Patator, the models trained on CIC17 showed very high MCC both in within-dataset and cross-dataset evaluation but the models trained on CIC18, although performed well in within-dataset configuration, obtained MCC less than 1% when evaluated on the external test set. The models trained with DDoS LOIC-HTTP attack samples showed near perfect generalization when trained on LycoS18 and tested on LycoS17. However, in the other 3 cross-dataset configurations their performances approached MCC 0%, with the only exception of LDA in the configuration CIC18 on CIC17 which achieved a 61.89% MCC. This phenomenon may be attributed to the possibility that, in one of the two datasets, the attack samples do not adequately represent the true distribution of that class. Instead, they might represent a subclass of the class distribution of the other dataset, leading to cross-dataset generalization occurring predominantly in one direction.

C. FEATURE SELECTION

In Figure 3 and Figure 4 are reported the results obtained by varying the number of features used for training with CIC and LycoS datasets, respectively. The experiments are performed in a binary manner using a malicious label that represents all the different attacks. The features employed in each experiment are reported in Table 5. Since mRMR is an incremental method, the optimal subset of n features is given by the first n features in one of the reported lists. Compared to the experiments employing the original feature set, also in this case within-dataset experiments achieved significantly better performance with respect to cross-dataset experiments with an average MCC of 83.86% and 14.91%, respectively.

As expected, performance in within-dataset experiments increases with the number of features, whereas in cross-dataset experiments we observe a different behavior. In 11 out of 16 experiments, the best result was achieved with a low number of features, within the range [3, 5]. This value is considerably low compared to the total original number of features, that is 77 both for CIC and LycoS datasets. This can be due to the fact that using a low number of features reduces overfitting, encouraging the models to learn more general patterns. 34 out of 64 experiments achieved MCC greater than 85% employing 4 or less features. Using only one feature, the average MCC among the within-dataset experiments is 69.7%, which corresponds to an accuracy of 91.22%. This indicates a strong correlation of some features with the label in the specific dataset. Unfortunately, these features are not sufficient to detect the attack since the performance in cross-dataset experiments drops sharply. We believe that the attacks are excessively homogeneous, allowing the attack samples of a single dataset to be represented using a minimal number of features.

TABLE 5. List of best features calculated with mRMR for binary experiments.

Dataset	Best 20 features
CIC17	Bwd Packet Length Std, ACK Flag Count, Fwd Packet Length Mean, Bwd Packet Length Max, Bwd Packet Length Mean, Avg Bwd Segment Size, Fwd IAT Std, Min Packet Length, Packet Length Variance, Packet Length Std, Idle Max, Max Packet Length, Bwd Packet Length Min, Idle Mean, Average Packet Size, Fwd IAT Max, Packet Length Mean, Flow IAT Max, Idle Min, Flow IAT Std
CIC18	min_seg_size_forward, Packet Length Mean, Bwd Packets/s, Init_Win_bytes_forward, ACK Flag Count, Fwd Packet Length Mean, Bwd Packet Length Min, Destination Port, Avg Fwd Segment Size, Min Packet Length, Init_Win_bytes_backward, Average Packet Size, Flow Packets/s, Fwd Packet Length Min, Total Fwd Packets, Bwd IAT Total, Fwd Packet Length Max, Bwd Packet Length Mean, Fwd PSH Flags, Avg Bwd Segment Size
LycoS17	pkt_len_std, fwd_non_empty_pkt_cnt, fwd_pkt_hdr_len_min, flag_rst, down_up_ratio, bwd_pkt_len_std, pkt_len_mean, ip_prot_6, pkt_len_var, bwd_pkt_len_mean, ip_prot_17, bwd_pkt_len_max, pkt_len_max, bwd_bulk_rate_mean, pkt_len_min, dst_port, bwd_pkt_hdr_len_min, flag_SYN, fwd_pkt_len_min, fwd_tcp_init_win_bytes
LycoS18	fwd_tcp_init_win_bytes, fwd_iat_tot, pkt_per_s, flag_fin, down_up_ratio, fwd_pkt_hdr_len_min, bwd_pkt_hdr_len_min, bwd_pkt_per_s, fwd_pkt_per_s, ip_prot_6, ip_prot_17, flag_SYN, fwd_pkt_cnt, dst_port, flow_duration, pkt_len_min, bwd_iat_tot, fwd_pkt_len_min, fwd_iat_std, flag_rst

Figures 5 and 6 show the results obtained by single-attack experiments using only the best two features listed in Table 6 for each combination of attack and training set. When examining the DoS Slowloris attack in the context of the CIC datasets, we note a degradation in within-dataset performance, coupled with an increase in cross-dataset performance. The Bot attack is not effectively classified in any of the configurations with the CIC datasets, whereas when using the LycoS datasets it is the second attack, alongside DoS Slowloris, to achieve similar within-dataset and cross-dataset performance. This discrepancy is likely attributed to errors in the labeling of Bot samples in the CIC datasets that have been corrected in the LycoS datasets.

D. VISUAL FEATURE SPACE ANALYSIS

Experiments using only two features confirm the trend already shown using the entire feature set, with high within-dataset and low cross-dataset performances. This provides a sound foundation for the visual analysis of the datasets in the 2D feature subspace.

Tables 7 and 8 report the counts of unique values for the two top features derived with mRMR for each attack type. Table 7 refers to features calculated using

TABLE 6. Best two features selected with mRMR for each attack and dataset.

Attack	CIC17	CIC18	LycoS17	LycoS18
DoS GoldenEye	Bwd Packet Length Std Flow IAT Mean	min_seg_size_forward Flow Bytes/s	bwd_pkt_len_std pkt_len_std	fwd_pkt_hdr_len_min fwd_tcp_init_win_bytes
DoS Slowloris	Bwd IAT Mean Init_Win_bytes_forward	Bwd IAT Mean Down/Up Ratio	fwd_pkt_hdr_len_min bwd_iat_mean	bwd_iat_mean fwd_pkt_hdr_len_min
DoS Hulk	Bwd Packet Length Std ACK Flag Count	min_seg_size_forward Packet Length Mean	pkt_len_mean flow_duration	fwd_pkt_hdr_len_min bwd_iat_min
DoS Slowhttptest	Active Mean Bwd IAT Std	Bwd Packets/s min_seg_size_forward	active_mean bwd_iat_std	pkt_per_s bwd_pkt_per_s
DDoS LOIC-HTTP	Bwd Packet Length Std Bwd Packet Length Mean	Avg Fwd Segment Size Fwd Header Length	pkt_len_mean bwd_iat_tot	flag_fin iat_std
SSH-Patator	URG Flag Count Flow IAT Mean	min_seg_size_forward Average Packet Size	fwd_flag_psh down_up_ratio	fwd_flag_psh iat_mean
FTP-Patator	Fwd PSH Flags Flow IAT Max	Bwd Packets/s min_seg_size_forward	down_up_ratio fwd_iat_min	pkt_per_s bwd_pkt_per_s
Bot	PSH Flag Count Fwd IAT Mean	RST Flag Count Idle Min	dst_port fwd_non_empty_pkt_cnt	dst_port flag_fin
Web Attack - Brute Force	Init_Win_bytes_backward ACK Flag Count	Fwd Packet Length Mean Destination Port	bwd_pkt_hdr_len_min down_up_ratio	fwd_flag_psh idle_max
Web Attack - XSS	Init_Win_bytes_backward ACK Flag Count	Total Length of Fwd Packets Fwd Header Length	bwd_pkt_hdr_len_min down_up_ratio	fwd_flag_psh flag_psh
Web Attack - Sql Injection	Bwd Packet Length Std min_seg_size_forward	Packet Length Variance Down/Up Ratio	bwd_pkt_len_std fwd_pkt_len_mean	flag_fin bwd_flag_psh
Infiltration	Flow Duration ACK Flag Count	Bwd Packets/s Fwd URG Flags		

LycoS17 as training set while Table 8 refers to features calculated using Lycos18 as training set. Comparing this information with those reported in Table 2 clearly shows strong sample redundancy for most classes in both datasets. Several attacks are represented by a single repeated instance. This is aggravated, but not due to the use of only two features. For instance, DoS Slowhttptest attack has only 56 different instances in the LycoS18 dataset using the entire feature set and a total number of samples of 105,550. The aforementioned conditions may negatively affect ML applications where the data should ideally exhibit sufficient variability.

Figure 7 depicts the distributions computed with KDE for each attack, utilizing the top two features derived from the LycoS17 dataset. All 11 attacks, with the exception of DoS Slowloris, show substantially different distributions for the two datasets. In some cases, such as DoS Hulk, DoS Slowhttptest and FTP-Patator, the distribution in the LycoS17 dataset is spread over multiple clusters while the counterpart is represented by a single point or a distribution with a significantly lower variance. It is also interesting to note that the DoS Slowhttptest attack in the LycoS18 dataset, despite having only one possible instance, is actually a subclass of the same attack in the LycoS17 dataset. Other cases, such as that of DoS GoldenEye, show similar shape distribution, but with a large difference in scale. As to DDoS LOIC-HTTP attack, the pkt_len_mean feature was selected as the most informative, and it is indeed an important parameter in the execution of this attack. It can be seen how in the LycoS18 dataset this parameter always takes exactly the value 140.571429, while in the Lycos17 dataset it takes different values distributed over 3 clusters. This indicates that a different configuration was employed when generating the DDoS LOIC-HTTP attack for the two datasets. Furthermore, by arbitrarily fixing a parameter during the attack execution, it is not possible to generate traffic samples that adequately represent that specific attack, as they depict only one potential instance. The only attack that has a similar representation in both datasets is DoS Slowloris. Hence the average reduction in performance between within-dataset and cross-dataset experiments is only 38.93%, whereas for the other attacks the average reduction is 84.36%.

To further confirm that the distribution of the classes between LycoS17 and LycoS18 datasets is substantially different, in Figure 8 we plot the KDE estimation of the attack distributions in a 2D subspace obtained with PCA fitted on LycoS17. Although the distributions in this different subspace are obviously different from those shown in Figure 7, analogous conclusions can be inferred. Specifically, it can be seen that for some attacks, namely DoS Slowhttptest, SSH-Patator, and FTP-Patator, the distribution in the LycoS17 dataset consists of one or more clusters with a higher variance

TABLE 7. Number of unique values for each of the two best features, named ϕ_1 and ϕ_2, calculated with mRMR using LycoS17 as training set (fourth column of Table 6). In bold are highlighted the attacks for which in both datasets the number of unique instances is less than 5% of the total.

	LycoS17		LycoS18	
	ϕ_1	ϕ_2	ϕ_1	ϕ_2
DoS GoldenEye	98	4,915	7	2,381
DoS Slowloris	3	2,891	2	6,415
DoS Hulk	2,899	82,383	1,908	378,266
DoS Slowhttptest	3,215	1,142	1	1
DDoS LOIC-HTTP	11	90,365	1	251,453
SSH-Patator	15	56	2	33
FTP-Patator	5	230	1	1
Bot	1	22	1	35
Web Attack-Brute Force	2	13	2	11
Web Attack - XSS	2	15	2	10
Web Attack - Sql Injection	9	10	37	38

TABLE 8. Number of unique values for each of the two best features, named ϕ_1 and ϕ_2, calculated with mRMR using LycoS18 as training set (fifth column of Table 6). In bold are highlighted the attacks for which in both datasets the number of unique instances is less than 5% of the total.

	LycoS17		LycoS18	
	ϕ_1	ϕ_2	ϕ_1	ϕ_2
DoS GoldenEye	2	1	1	2
DoS Slowloris	2,891	3	6,415	2
DoS Hulk	2	829	1	48,024
DoS Slowhttptest	1,492	1,076	56	56
DDoS LOIC-HTTP	2	95,683	1	289,327
SSH-Patator	15	2,956	2	77,713
FTP-Patator	3,988	3,973	54	54
Bot	1	1	1	2
Web Attack-Brute Force	10	1	7	1
Web Attack - XSS	7	7	6	7
Web Attack - Sql Injection	1	1	1	4

with respect to distribution in the LycoS18 dataset that is concentrated around a single point. Differently from Figure 7, the distributions of DoS Hulk attack between the two datasets exhibit partial overlap. Hence, the mRMR method, seeing only the information of the training set, has selected an optimal 2D subspace that exacerbates the difference between the two distributions. Also in this 2D subspace, all the attack classes are differently distributed among the two datasets except in the case of DoS Slowloris.

V. CONCLUSION

This work focuses on analyzing the generalization capabilities of ML-based NIDS using different datasets for training and evaluating the models. We found that training a ML model on a single publicly available dataset is not sufficient to create a robust NIDS capable of working in different datasets and real-world applications.

The main motivations behind this conclusion are related to dataset composition. The classes in the four datasets employed do not adequately represent the real distribution of attacks, limiting them to representing the particular instance of the attack that was executed when the dataset was created and thus dependent on specific tools and parameters used. Additionally, we found that in some cases only a few features are needed to correctly identify an attack. This indicates a dataset-specific overcorrelation with the label that prevents the model from learning the true attack pattern. Other possible motivations are related to the difficulties in correctly labeling an attack and other factors that influence the distribution of samples in the feature space, such as the topology of the networks and the hardware used for communication.

In the literature, many works utilize federated learning to address the generalization problem of ML-based NIDS, demonstrating promising results. However, unlike our methodology, these models were exposed to data from all the diverse sources employed in the testing phase, potentially biasing the evaluation of their generalization capability. Future research could explore the potential of federated learning using a sufficient number of diverse datasets, where each local model is trained on a single dataset, and the global model is tested on a new dataset. This approach may enhance the heterogeneity of the training set, yielding enhanced generalization potential. Additionally, more powerful deep learning models that learn features directly from raw PCAP data could be used to extract network-invariant features, addressing the limitations of traditional handcrafted features. Training models capable of identifying anomalous traffic in different network contexts presents considerable difficulties. This holds in particular when considering the necessity to construct datasets that thoroughly and adequately represent the highly heterogeneous nature of network traffic. These challenges cast doubts on the practicality of employing ML for designing NIDS.

ACKNOWLEDGMENT

The authors utilized ChatGPT for English language proofreading. Following the use of this tool, the authors reviewed and edited the content as necessary, taking full responsibility for the final publication.

REFERENCES

[1] F. Cremer, B. Sheehan, M. Fortmann, A. N. Kia, M. Mullins, F. Murphy, and S. Materne, "Cyber risk and cybersecurity: A systematic review of data availability," *Geneva Papers Risk Insurance-Issues Pract.*, vol. 47, no. 3, pp. 698–736, Jul. 2022.

[2] J. M. Torres, C. I. Comesa na, and P. J. García-Nieto, "Machine learning techniques applied to cybersecurity," *Int. J. Mach. Learn. Cybern.*, vol. 10, no. 10, pp. 2823–2836, 2019.

[3] D. Dasgupta, Z. Akhtar, and S. Sen, "Machine learning in cybersecurity: A comprehensive survey," *J. Defense Model. Simul., Appl., Methodol, Technol.*, vol. 19, no. 1, pp. 57–106, Jan. 2022.

[4] I. H. Sarker, A. S. M. Kayes, S. Badsha, H. Alqahtani, P. Watters, and A. Ng, "Cybersecurity data science: An overview from machine learning perspective," *J. Big Data*, vol. 7, no. 1, pp. 1–29, Dec. 2020.

[5] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network anomaly detection: Methods, systems and tools," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 1, pp. 303–336, 1st Quart., 2014.

[6] M. Alkasassbeh and S. A.-H. Baddar, "Intrusion detection systems: A state-of-the-art taxonomy and survey," *Arabian J. Sci. Eng.*, vol. 48, no. 8, pp. 10021–10064, Aug. 2023.

[7] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, Dec. 2019.

[8] Z. Tan, A. Jamdagni, X. He, P. Nanda, and R. P. Liu, "A system for denial-of-service attack detection based on multivariate correlation analysis," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 2, pp. 447–456, Feb. 2014.

[9] J. Camacho, A. Pérez-Villegas, P. García-Teodoro, and G. Maciá-Fernández, "PCA-based multivariate statistical network monitoring for anomaly detection," *Comput. Secur.*, vol. 59, pp. 118–137, Jun. 2016.

[10] R. SakilaAnnarasi and S. Sivanesh, "A secure intrusion detection system for MANETs," in *Proc. IEEE Int. Conf. Adv. Commun., Control Comput. Technol.*, May 2014, pp. 1174–1178.

[11] G. R. Hendry and S. J. Yang, "Intrusion signature creation via clustering anomalies," *Proc. SPIE*, vol. 6973, pp. 119–130, Mar. 2008.

[12] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Müller, and K. Hanssgen, "A survey of payload-based traffic classification approaches," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 2, pp. 1135–1156, 2nd Quart., 2014.

[13] M. M. Raikar, S. M. Meena, M. M. Mulla, N. S. Shetti, and M. Karanandi, "Data traffic classification in software defined networks (SDN) using supervised-learning," *Proc. Comput. Sci.*, vol. 171, pp. 2750–2759, Jan. 2020.

[14] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection," *Big Data Res.*, vol. 30, Nov. 2022, Art. no. 100359.

[15] M. Verkerken, L. D'Hooge, D. Sudyana, Y.-D. Lin, T. Wauters, B. Volckaert, and F. D. Turck, "A novel multi-stage approach for hierarchical intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 3915–3929, Sep. 2023.

[16] H. Choi, M. Kim, G. Lee, and W. Kim, "Unsupervised learning approach for network intrusion detection system using autoencoders," *J. Supercomput.*, vol. 75, no. 9, pp. 5597–5621, Sep. 2019.

[17] M. Verkerken, L. D'Hooge, T. Wauters, B. Volckaert, and F. De Turck, "Unsupervised machine learning techniques for network intrusion detection on modern data," in *Proc. 4th Cyber Secur. Netw. Conf. (CSNet)*, Oct. 2020, pp. 1–8.

[18] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *Appl. Sci.*, vol. 9, no. 20, p. 4396, Oct. 2019.

[19] T. Saranya, S. Sridevi, C. Deisy, T. D. Chung, and M. K. A. A. Khan, "Performance analysis of machine learning algorithms in intrusion detection system: A review," *Proc. Comput. Sci.*, vol. 171, pp. 1251–1260, Jan. 2020.

[20] Y. Zhou, G. Cheng, S. Jiang, and M. Dai, "Building an efficient intrusion detection system based on feature selection and ensemble classifier," *Comput. Netw.*, vol. 174, Jun. 2020, Art. no. 107247.

[21] D. Stiawan, Darmawijoyo, M. Y. B. Idris, A. M. Bamhdi, and R. Budiarto, "CICIDS-2017 dataset feature analysis with information gain for anomaly detection," *IEEE Access*, vol. 8, pp. 132911–132921, 2020.

[22] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve Bayes feature embedding," *Comput. Secur.*, vol. 103, Apr. 2021, Art. no. 102158.

[23] S. Mahdavifar and A. A. Ghorbani, "Application of deep learning to cybersecurity: A survey," *Neurocomputing*, vol. 347, pp. 149–176, Jun. 2019.

[24] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 41–50, Feb. 2018.

[25] G. Marín, P. Casas, and G. Capdehourat, "RawPower: Deep learning based anomaly detection from raw network traffic measurements," in *Proc. ACM SIGCOMM Conf. Posters Demos*, Aug. 2018, pp. 75–77.

[26] T. Saba, A. Rehman, T. Sadad, H. Kolivand, and S. A. Bahaj, "Anomaly-based intrusion detection system for IoT networks through deep learning model," *Comput. Electr. Eng.*, vol. 99, Apr. 2022, Art. no. 107810.

[27] E. Mushtaq, A. Zameer, M. Umer, and A. A. Abbasi, "A two-stage intrusion detection system with auto-encoder and LSTMs," *Appl. Soft Comput.*, vol. 121, May 2022, Art. no. 108768.

[28] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 699–714, 2023.

[29] X. Li, W. Chen, Q. Zhang, and L. Wu, "Building auto-encoder intrusion detection system based on random forest feature selection," *Comput. Secur.*, vol. 95, Aug. 2020, Art. no. 101851.

[30] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.

[31] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.

[32] W. Jo, S. Kim, C. Lee, and T. Shon, "Packet preprocessing in CNN-based network intrusion detection system," *Electronics*, vol. 9, no. 7, p. 1151, Jul. 2020.

[33] G. Agrafiotis, E. Makri, I. Flionis, A. Lalas, K. Votis, and D. Tzovaras, "Image-based neural network models for malware traffic classification using PCAP to picture conversion," in *Proc. 17th Int. Conf. Availability, Rel. Secur.*, Aug. 2022, pp. 1–7.

[34] C. M. K. Ho, K.-C. Yow, Z. Zhu, and S. Aravamuthan, "Network intrusion detection via flow-to-image conversion and vision transformer classification," *IEEE Access*, vol. 10, pp. 97780–97793, 2022.

[35] Y. Li, X. Yuan, and W. Li, "An extreme semi-supervised framework based on transformer for network intrusion detection," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2022, pp. 4204–4208.

[36] M. Li, D. Han, D. Li, H. Liu, and C.-C. Chang, "MFVT: An anomaly traffic detection method merging feature fusion network and vision transformer architecture," *EURASIP J. Wireless Commun. Netw.*, vol. 2022, no. 1, p. 39, Dec. 2022.

[37] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "RTIDS: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64375–64387, 2022.

[38] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSp*, vol. 1, pp. 108–116, Jan. 2018.

[39] A. Rosay, E. Cheval, F. Carlier, and P. Leroux, "Network intrusion detection: A comprehensive analysis of CIC-IDS2017," in *Proc. 8th Int. Conf. Inf. Syst. Secur. Privacy*, 2022, pp. 25–36.

[40] Y. Yang, H. Fu, S. Gao, Y. Zhou, and W. Shi, "Intrusion detection: A model based on the improved vision transformer," *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 9, pp. 185–205, Sep. 2022.

[41] S. Ustebay, Z. Turgut, and M. A. Aydin, "Intrusion detection system with recursive feature elimination by using random forest and deep learning classifier," in *Proc. Int. Congr. Big Data, Deep Learn. Fighting Cyber Terrorism (IBIGDELFT)*, Dec. 2018, pp. 71–76.

[42] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A deep learning approach for network intrusion detection system," in *Proc. 9th EAI Int. Conf. Bio-Inspired Inf. Commun. Technol.*, 2016, pp. 21–26.

[43] S. Agrawal, S. Sarkar, O. Aouedi, G. Yenduri, K. Piamrat, M. Alazab, S. Bhattacharya, P. K. R. Maddikunta, and T. R. Gadekallu, "Federated learning for intrusion detection system: Concepts, challenges and future directions," *Comput. Commun.*, vol. 195, pp. 346–361, Nov. 2022.

[44] L. D'Hooge, T. Wauters, B. Volckaert, and F. De Turck, "Inter-dataset generalization strength of supervised machine learning methods for intrusion detection," *J. Inf. Secur. Appl.*, vol. 54, Oct. 2020, Art. no. 102564.

[45] M. Verkerken, L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Towards model generalization for intrusion detection: Unsupervised machine learning techniques," *J. Netw. Syst. Manage.*, vol. 30, no. 1, pp. 1–25, Jan. 2022.

[46] I. Sič ić, N. Petrović, K. Slovenec, and M. Mikuc, "Evaluation of inter-dataset generalisability of autoencoders for network intrusion detection," in *Proc. 17th Int. Conf. Telecommun. (ConTEL)*, Jul. 2023, pp. 1–7.

[47] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Netw. Appl.*, vol. 27, no. 1, pp. 357–370, Feb. 2022.

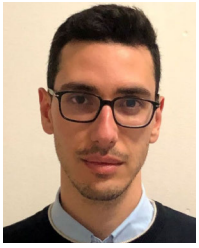
[48] S. I. Popoola, G. Gui, B. Adebisi, M. Hammoudeh, and H. Gacanin, "Federated deep learning for collaborative intrusion detection in heterogeneous networks," in *Proc. IEEE 94th Veh. Technol. Conf. (VTC-Fall)*, Sep. 2021, pp. 1–6.

[49] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "Cyber threat intelligence sharing scheme based on federated learning for network intrusion detection," *J. Netw. Syst. Manage.*, vol. 31, no. 1, p. 3, Jan. 2023.

[50] G. de Carvalho Bertoli, L. A. Pereira Jr., O. Saotome, and A. L. dos Santos, "Generalizing intrusion detection for heterogeneous networks: A stacked-unsupervised federated learning approach," *Comput. Secur.*, vol. 127, Apr. 2023, Art. no. 103106.

[51] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 7–12.

[52] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Proc. Int. Conf. Risks Secur. Internet Syst.* Cham, Switzerland: Springer, 2022, pp. 18–33.
[53] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Oct. 2022, pp. 254–262.
[54] M. Catillo, A. Pecchia, and U. Villano, "Machine learning on public intrusion datasets: Academic hype or concrete advances in NIDS?" in *Proc. 53rd Annu. IEEE/IFIP Int. Conf. Dependable Syst.*, Jun. 2023, pp. 132–136.
[55] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023.
[56] Y. Kim, S. Hakak, and A. Ghorbani, "DDoS attack dataset (CICEV2023) against EV authentication in charging infrastructure," in *Proc. 20th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2023, pp. 1–9.
[57] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol. (ICCST)*, Oct. 2019, pp. 1–8.
[58] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018, *arXiv:1802.09089*.
[59] N. Farnaaz and M. A. Jabbar, "Random forest modeling for network intrusion detection system," in *Proc. Comput. Sci.*, vol. 89, pp. 213–217, Jan. 2016.
[60] S. Waskle, L. Parashar, and U. Singh, "Intrusion detection system using PCA with random forest approach," in *Proc. Int. Conf. Electron. Sustain. Commun. Syst. (ICESC)*, Jul. 2020, pp. 803–808.
[61] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, vol. 11, Aug. 2016, pp. 785–794.
[62] P. Devan and N. Khare, "An efficient XGBoost–DNN-based classification model for network intrusion detection system," *Neural Comput. Appl.*, vol. 32, no. 16, pp. 12499–12514, Aug. 2020.
[63] A. Husain, A. Salem, C. Jim, and G. Dimitoglou, "Development of an efficient network intrusion detection model using extreme gradient boosting (XGBoost) on the UNSW-NB15 dataset," in *Proc. IEEE Int. Symp. Signal Process. Inf. Technol. (ISSPIT)*, Dec. 2019, pp. 1–7.
[64] B. S. Bhati, G. Chugh, F. Al-Turjman, and N. S. Bhati, "An improved ensemble based intrusion detection technique usingXGBoost," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 6, p. e4076., Jun. 2021.
[65] C. Ding and H. Peng, "Minimum redundancy feature selection from microarray gene expression data," *J. Bioinf. Comput. Biol.*, vol. 3, no. 2, pp. 185–205, Apr. 2005.
[66] H. Peng, F. Long, and C. Ding, "Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 8, pp. 1226–1238, Aug. 2005.
[67] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 1, pp. 1–13, Dec. 2020.
[68] J. Yao and M. Shepperd, "Assessing software defection prediction performance: Why using the Matthews correlation coefficient matters," in *Proc. Eval. Assessment Softw. Eng.*, Apr. 2020, pp. 120–129.
[69] S. Boughorbel, F. Jarray, and M. El-Anbari, "Optimal classifier for imbalanced data using Matthews correlation coefficient metric," *PLoS ONE*, vol. 12, no. 6, Jun. 2017, Art. no. e0177678.



MARCO CANTONE received the bachelor's degree in computer science and telecommunications engineering and the master's degree in computer engineering from the University of Cassino and Southern Lazio, in 2019 and 2022, respectively, where he is currently pursuing the Ph.D. degree in artificial intelligence with the Department of Electrical and Information Engineering (DIEI). His research interests include neural networks, pattern recognition, and computer vision, with applications in various fields such as healthcare and cybersecurity.

CLAUDIO MARROCCO (Member, IEEE) is currently working as an Associate Professor of computer science and artificial intelligence with the Department of Electrical and Information Engineering (DIEI), University of Cassino and Southern Lazio, Italy. He has authored more than 70 papers in international top journals and conference proceedings. His research interests include pattern recognition and image processing and analysis, with a particular attention to machine and deep learning techniques for unbalanced data and multiple expert systems applied to medical images, neuroscience, cultural heritage, and the Internet of Things.

ALESSANDRO BRIA received the M.Sc. degree in computer engineering from the University of L'Aquila, in 2010, and the Ph.D. degree in electrical and information engineering from the University of Cassino and Southern Lazio, in 2014. In 2019, he joined the Department of Electrical and Information Engineering (DIEI), University of Cassino and Southern Lazio, where he is currently working as an Associate Professor in computer science and artificial intelligence. He has authored 60 International journals and conference proceedings research papers. His current research interests include medical image analysis, EEG signal analysis, and unstructured EHR analysis. He is a member of the International Association of Pattern Recognition (IAPR). He is a member of the Editorial Board of the journals *Frontiers in Artificial Intelligence* and *Frontiers in Big Data*.

Open Access funding provided by 'Università degli Studi di Cassino e del Lazio Meridionale' within the CRUI CARE Agreement